

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

BASE DE DATOS II

Unidad 1 · Integridad, Transacciones y Concurrency

Trabajo Práctico de Laboratorio: Concurrency e IA como motor primario

ENTREGA COMPLETA — Partes 0, 1, 2 y 3

Proyecto integrador	Food Store (esquema de la Semana 1)
Motor	PostgreSQL 17.10
Cliente	DBeaver / psql (dos sesiones concurrentes)
Base de trabajo	foodstore_tp2 (copia según protocolo, nunca foodstore)
Herramientas de IA	OpenCode (agente CLI) + Kiro — delegación de escritura, nunca de decisión
Integrantes	Guillot Thiago · Monardez Máximo · Ruiz Matías · Méndez Joaquín · Oviedo Thiago · Chacón Facundo

Podés explicar, sin mirar el archivo, qué hace cada línea que commiteaste.

Índice

1. Consigna del TP2 (enunciado de cátedra)
2. Parte 0 — Protocolo de seguridad (protocolo_seguridad.md)
3. Parte 1 — Integridad versionada: spec, script, pruebas y DUIA
4. Parte 2 — Laboratorio de concurrencia: informe, evidencia del motor y DUIA
5. Parte 3 — Lectura crítica: análisis y corrección (ejercicio_lectura_critica.md y DUIA)
6. Historial de commits (git log)
7. Checklist final antes de entregar

1. Consigna del TP2 — Resumen

Área	Consigna
Objetivos	Reproducir con dos sesiones concurrentes al menos tres de las cuatro anomalías de la Semana 2; generar restricciones de integridad con IA versionadas en Git; aplicar el protocolo de seguridad (copia, transacción, respaldo) en cada paso; practicar lectura crítica de un script deliberadamente peligroso; y completar una DUIA por ejercicio.
Requisitos previos	Esquema propio (schema.sql) sobre una base local; repo Git con commit inicial; OpenCode y Kiro configurados; dos conexiones a la base para simular dos sesiones.
Regla de fondo	Se delega la escritura, nunca la decisión. Todo script generado se lee, se prueba sobre una copia dentro de una transacción, y se defiende oralmente antes de darlo por bueno.
Parte 0	Protocolo de seguridad adaptado al entorno real, commiteado antes de continuar.
Parte 1	Dos a tres reglas de negocio garantizadas en el motor (triggers) + DUIA + defensa oral del diff.
Parte 2	Mínimo tres escenarios de concurrencia sobre el propio esquema, con verificación de la explicación de la IA en el motor real + informe + DUIA.
Parte 3	Análisis y corrección de dos scripts peligrosos (UPDATE sin WHERE, NOT IN con NULL) + DUIA.

2. Parte 0 — Protocolo de seguridad

Protocolo de Seguridad — TP2 Laboratorio de Concurrency e IA

Materia: Base de Datos II — Tecnicatura Universitaria en Programación (UTN) TP: 2 · Concurrency e IA como motor primario (Unidad 1, Semana 2) Motor: PostgreSQL 17.10 · Cliente: DBeaver / psql Integrantes: Guillot Tiago, Monardez Maximo, Ruiz Matias, Mendez Joaquin, Oviedo Thiago y Chacon Facundo Proyecto integrador: Food Store (`schema.sql` de la Semana 1)

Regla de fondo

Se delega la escritura, nunca la decisión.

Todo script —propio o generado por una IA— que toque la base se aplica siempre siguiendo este protocolo de tres pasos. No es una excepción que se salta cuando el cambio "parece trivial": un `UPDATE` sin `WHERE` no parece riesgoso hasta que borra todo.

Este documento adapta los tres pasos de la cátedra al entorno real del proyecto (PostgreSQL 17 local, base `foodstore`, respaldo con `pg_dump`).

Los tres pasos: copia → transacción → respaldo

Paso 1 — Copia (siempre sobre una base de desarrollo)

Nunca se trabaja sobre la base que contiene datos que importan. La base real de trabajo de Semana 1 es `foodstore`; para el TP2 se trabaja sobre una copia creada a partir de una plantilla.

```
# 1. Crear la copia de trabajo usando la base real como plantilla
createdb -U postgres -h localhost -T foodstore foodstore_tp2

# 2. Verificar que la copia tiene el mismo esquema y los mismos datos
psql -U postgres -h localhost -d foodstore_tp2 -c "\dt"
```

Cuándo se salta: no aplica. Siempre hay copia. Si no existiera la base plantilla, se carga `schema.sql` en una base vacía nueva.

Nota de entorno: `createdb -T` con copia template necesita que nadie esté conectado a la base origen al momento de copiarla (`foodstore` debe estar sin conexiones activas). Si está en uso, usar `pg_dump` + `pg_restore` como alternativa de copia.

Paso 2 — Transacción (todo script de escritura empieza con `BEGIN`)

Todo script que escribe corre primero dentro de una transacción que se revierte, para inspeccionar el efecto (filas afectadas, mensajes de error) antes de confirmar nada.

```
BEGIN;

-- ... script de escritura (INSERT / UPDATE / DELETE / DDL) ...
-- ... prueba con casos válidos e inválidos ...

-- Inspeccionar el efecto ANTES de confirmar:
SELECT * FROM <tabla> WHERE ...;

-- Si algo salió mal, deshacer todo:
ROLLBACK;

-- Solo cuando se entiende y verifica el efecto:
COMMIT;
```

Cuándo se salta: no aplica. Siempre BEGIN antes de escribir. El ROLLBACK es la red de seguridad que permite que un error propio —o de la IA— no toque nada real.

Paso 3 — Respaldo (obligatorio antes de cualquier cambio estructural)

Antes de aplicar un cambio estructural (`ALTER TABLE`, `DROP`, una migración o un trigger nuevo), se hace un respaldo independiente. El `ROLLBACK` del Paso 2 alcanza para una transacción puntual, pero no para decisiones estructurales que conviene poder volver atrás sin depender de la transacción.

```
# Respaldo lógico completo de la copia de trabajo, con marca de fecha/hora
pg_dump -U postgres -h localhost -d foodstore_tp2 \
  --file="C:\UTN\3er semestre\Bases de Datos II\UNIDAD 1\backups\foodstore_tp2_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').sql"

# Listar respaldos disponibles
Get-ChildItem "C:\UTN\3er semestre\Bases de Datos II\UNIDAD 1\backups"
```

Cuándo se salta: no aplica. Siempre respaldo antes de DDL.

Lugar donde vive el respaldo: carpeta backups/ dentro del directorio del proyecto, que se conserva localmente (está excluida del repositorio Git por .gitignore, porque es un artefacto regenerable vía pg_dump). Se usa pg_dump (respaldo lógico) porque es portable y legible.

Checklist que se repite en CADA operación

- [] Trabajo sobre la copia `foodstore_tp2`, nunca sobre `foodstore`.
 - [] Respaldo `pg_dump` tomado antes de cualquier DDL.
 - [] Script leído línea por línea antes de ejecutarlo (ningún script de IA se corre sin leerse).
 - [] Script aplicado dentro de `BEGIN ... ROLLBACK` y verificado antes del `COMMIT`.
 - [] Si era script de IA: `git diff` revisado línea a línea y DUIA completada.
-

Dónde vive este protocolo

Este archivo (`protocolo_seguridad.md`) se commitea en la raíz del repo antes de avanzar a la Parte 1. Sin este archivo no se continúa.

3. Parte 1 — Integridad versionada

3.1. Spec de las restricciones (se escribe antes del script)

Spec — Restricciones de integridad para Food Store (Parte 1)

Regla de fondo: _se delega la escritura, nunca la decisión_. Esta spec se escribe antes de generar el script, porque una spec ambigua produce un script ambiguo.

Motor objetivo: PostgreSQL 17 · Base de trabajo: foodstore_tp2 (copia) Tablas involucradas: cliente, producto, pedido, linea_pedido

Regla 1 — Clientes inactivos no pueden generar pedidos nuevos

- Tabla/columna: se valida en `pedido` al insertar, leyendo `cliente.activo`.
- Enunciado: un pedido nuevo no puede pertenecer a un cliente dado de baja (`cliente.activo = FALSE`).
- Por qué: el esquema usa baja lógica (R7). Si un cliente está inactivo, no debe seguir operando (generar compras). Hoy esto lo valida la aplicación; con el trigger queda garantizado en el motor.
- Comportamiento esperado: `INSERT INTO pedido (cliente_id, forma_pago) ...` para un cliente inactivo → error.

Regla 2 — Productos inactivos no se pueden vender en líneas nuevas

- Tabla/columna: se valida en `linea_pedido` al insertar, leyendo `producto.activo`.
- Enunciado: una línea de pedido nueva no puede referenciar un producto dado de baja (`producto.activo = FALSE`).
- Por qué: la baja lógica (R7) implica que un producto retirado no sigue vendiéndose. Requiere un trigger porque combina dos tablas.
- Comportamiento esperado: `INSERT INTO linea_pedido` con un `producto_id` inactivo → error.

Regla 3 — La cantidad vendida no puede superar el stock disponible

- Tabla/columna: se valida en `linea_pedido` al insertar y al actualizar `cantidad`, comparando con `producto.stock`.
 - Enunciado: `linea_pedido.cantidad <= producto.stock` para el producto de esa línea.
 - Por qué: control de stock (R5: `stock >= 0`). Evita vender más unidades de las que hay en el depot. Requiere trigger porque interviene `producto.stock`.
 - Comportamiento esperado: `INSERT/UPDATE` de una línea con `cantidad > producto.stock` → error.
-

Alcance / fuera de alcance

- Dentro: triggers `BEFORE INSERT/BEFORE UPDATE` y `RAISE EXCEPTION` con mensajes descriptivos.
- Fuera: actualización automática de `stock` al vender (eso es lógica de negocio transaccional, no una restricción de integridad; se documenta como decisión).
- Fuera: descontar stock automáticamente. Se mantiene solo la regla que impide vender de más.

3.2. Script generado con OpenCode y revisado línea por línea (restricciones_integridad.sql)

```
-- =====
-- Food Store - Restricciones de integridad de negocio (TP2, Parte 1)
-- Generado con asistencia de IA (OpenCode) y REVISADO linea por linea por el alumno.
-- Motor: PostgreSQL 17 | Base de trabajo: foodstore_tp2 (copia, ver protocolo)
-- Reglas (spec en spec_restricciones.md):
--   R1: un cliente inactivo no puede generar pedidos nuevos.
--   R2: un producto inactivo no se puede vender en lineas de pedido nuevas.
--   R3: la cantidad vendida no puede superar el stock disponible del producto.
-- Aplicar dentro de una transaccion: BEGIN; ... pruebas ... COMMIT;
-- =====

-- -----
-- R1: no insertar pedidos para clientes inactivos
-- -----

CREATE OR REPLACE FUNCTION chk_activo_cliente_pedido() RETURNS trigger AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM cliente WHERE id = NEW.cliente_id AND activo = TRUE
    ) THEN
        RAISE EXCEPTION 'No se puede crear un pedido para el cliente % (inactivo o inexistente).',
            NEW.cliente_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_chk_activo_cliente_pedido ON pedido;
CREATE TRIGGER trg_chk_activo_cliente_pedido
    BEFORE INSERT ON pedido
    FOR EACH ROW
    EXECUTE FUNCTION chk_activo_cliente_pedido();

-- -----
-- R2: no vender en lineas nuevas productos inactivos
-- -----

CREATE OR REPLACE FUNCTION chk_activo_producto_linea() RETURNS trigger AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM producto WHERE id = NEW.producto_id AND activo = TRUE
    ) THEN
        RAISE EXCEPTION 'No se puede vender el producto % (inactivo o inexistente).',
            NEW.producto_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_chk_activo_producto_linea ON linea_pedido;
CREATE TRIGGER trg_chk_activo_producto_linea
    BEFORE INSERT ON linea_pedido
    FOR EACH ROW
    EXECUTE FUNCTION chk_activo_producto_linea();

-- -----
-- R3: la cantidad vendida no supera el stock disponible
-- -----

CREATE OR REPLACE FUNCTION chk_stock_linea() RETURNS trigger AS $$
DECLARE
    v_stock INTEGER;
BEGIN
    SELECT stock INTO v_stock FROM producto WHERE id = NEW.producto_id;
    IF v_stock IS NOT NULL AND NEW.cantidad > v_stock THEN
        RAISE EXCEPTION 'Stock insuficiente para el producto %: pide % pero hay % disponibles.',
            NEW.producto_id, NEW.cantidad, v_stock;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_chk_stock_linea ON linea_pedido;
CREATE TRIGGER trg_chk_stock_linea
```

```

BEFORE INSERT OR UPDATE OF cantidad ON linea_pedido
FOR EACH ROW
EXECUTE FUNCTION chk_stock_linea();

```

3.3. Pruebas de verificación: casos válidos e inválidos dentro de una transacción (pruebas_restricciones.sql)

```

-- =====
-- Pruebas de verificación de las restricciones de integridad (Parte 1)
-- Ejecutado sobre foodstore_tp2 (copia) dentro de UNA transaccion.
-- El DDL se incluye via \i, las pruebas usan bloques DO para capturar
-- los errores esperados (RAISE EXCEPTION de los triggers) sin abortar.
-- =====

BEGIN;

-- 1) Aplicar las restricciones (DDL transaccional)
\i restricciones_integridad.sql

-- -----
-- REGLA 1: cliente inactivo no genera pedidos
-- -----

UPDATE cliente SET activo = FALSE WHERE id = 1;
SELECT 'R1: cliente 1 dado de baja' AS paso;

DO $$
BEGIN
    BEGIN
        BEGIN
            INSERT INTO pedido (cliente_id, forma_pago) VALUES (1, 'EFFECTIVO');
            RAISE NOTICE 'R1-INVALIDO = FALLO (el trigger no rechazo, BUG)';
            EXCEPTION WHEN others THEN
                RAISE NOTICE 'R1-INVALIDO OK: pedido a cliente inactivo rechazado -> %', SQLERRM;
            END;
        END;
    END;
END $$;

UPDATE cliente SET activo = TRUE WHERE id = 1;
SELECT 'R1: cliente 1 reactivado' AS paso;

DO $$
BEGIN
    INSERT INTO pedido (cliente_id, forma_pago) VALUES (1, 'EFFECTIVO');
    RAISE NOTICE 'R1-VALIDO OK: pedido a cliente activo aceptado';
END $$;

-- -----
-- REGLA 2: producto inactivo no se vende en lineas nuevas
-- (pedido de prueba creado arriba; usar lastval para no chocar con PK)
-- -----

UPDATE producto SET activo = FALSE WHERE id = 2;
SELECT 'R2: producto 2 (Coca) dado de baja' AS paso;

DO $$
DECLARE
    v_pedido BIGINT := (
        SELECT max(id) FROM pedido
    );
BEGIN
    BEGIN
        BEGIN
            INSERT INTO linea_pedido (pedido_id, producto_id, cantidad, precio_unitario)
            VALUES (v_pedido, 2, 1, 800.00);
            RAISE NOTICE 'R2-INVALIDO = FALLO (el trigger no rechazo, BUG)';
            EXCEPTION WHEN others THEN
                RAISE NOTICE 'R2-INVALIDO OK: linea con producto inactivo rechazada -> %', SQLERRM;
            END;
        END;
    END;
END $$;

UPDATE producto SET activo = TRUE WHERE id = 2;
SELECT 'R2: producto 2 reactivado' AS paso;

```



```

DO $$
DECLARE
    v_pedido BIGINT := (SELECT max(id) FROM pedido);
BEGIN
    INSERT INTO linea_pedido (pedido_id, producto_id, cantidad, precio_unitario)
    VALUES (v_pedido, 1, 1, 1050.00);
    RAISE NOTICE 'R2-VALIDO OK: linea con producto activo aceptada';
END $$;

-- -----
-- REGLA 3: cantidad no superior al stock
-- -----

DO $$
DECLARE
    v_pedido BIGINT := (SELECT max(id) FROM pedido);
BEGIN
    BEGIN
        BEGIN
            -- producto 1 (Muzzarella) stock = 20
            INSERT INTO linea_pedido (pedido_id, producto_id, cantidad, precio_unitario)
            VALUES (v_pedido, 1, 5000, 1050.00);
            RAISE NOTICE 'R3-INVALIDO = FALLO (el trigger no rechaza, BUG)';
            EXCEPTION WHEN others THEN
                RAISE NOTICE 'R3-INVALIDO OK: cantidad sobre stock rechazada -> %', SQLERRM;
        END;
    END;
END $$;

DO $$
DECLARE
    v_pedido BIGINT := (SELECT max(id) FROM pedido);
BEGIN
    -- producto 2 (Coca) stock = 50, aun sin linea en este pedido
    INSERT INTO linea_pedido (pedido_id, producto_id, cantidad, precio_unitario)
    VALUES (v_pedido, 2, 3, 800.00);
    RAISE NOTICE 'R3-VALIDO OK: cantidad dentro del stock aceptada';
END $$;

-- -----
-- Inspeccion del efecto antes de decidir COMMIT o ROLLBACK
-- -----

SELECT 'Estado despues de las pruebas (transaccion aun abierta):' AS estado;
SELECT p.id AS pedido, count(lp.producto_id) AS lineas
FROM pedido p
LEFT JOIN linea_pedido lp ON lp.pedido_id = p.id
GROUP BY p.id
ORDER BY p.id;

-- Comentario de cierre: si todo se ve correcto, correr COMMIT;
-- si algo fallo, ROLLBACK; (la ejecucion manual decide).
ROLLBACK;
SELECT 'ROLLBACK aplicado: la transaccion de prueba NO persistio cambios' AS fin;

```

3.4. Declaración de Uso de IA — Parte 1

Declaración de Uso de IA (DUIA) — Parte 1 · Integridad versionada

TP: 2 · Concurrencia e IA como motor primario (Parte 1) Esquema: Food Store (proyecto integrador) Motor: PostgreSQL 17.10 · Base de trabajo: foodstore_tp2 (copia según protocolo)

Campo	Completar
Herramienta	OpenCode (modelo/proveedor configurado: modelo de línea big-pickle)

Spec o prompt utilizado	"Generará restricciones de integridad (triggers declarativos) para las 3 reglas de negocio de la spec_restricciones.md: (R1) cliente inactivo no genera pedidos, (R2) producto inactivo no se vende en líneas nuevas, (R3) cantidad vendida <= stock. Motor PostgreSQL 17, sobre el esquema FoodStore. Usá triggers BEFORE INSERT/UPDATE y RAISE EXCEPTION con mensajes descriptivos."
Qué generó	spec_restricciones.md (spec), restricciones_integridad.sql (3 funciones + 3 triggers: chk_activo_cliente_pedido, chk_activo_producto_linea, chk_stock_linea), pruebas_restricciones.sql (script de verificación)
Qué se aceptó	Las 3 funciones y los 3 triggers tal como se generaron, tras revisar el diff. La lógica de cada regla y los nombres de los objetos se mantuvieron.
Qué se modificó o descartado, y por qué	1) En pruebas_restricciones.sql, el caso válido de la R3 usaba inicialmente producto_id = 1 (Muzzarella), pero ese producto ya tenía una línea en el mismo pedido, lo que violaba la PK compuesta (pedido_id, producto_id) antes de llegar al trigger de stock. Se cambió a producto_id = 2 (Coca) para aislar la prueba del control de stock. 2) Los mensajes de error se ajustaron para incluir los valores concretos (id de cliente/producto, cantidades) para facilitar el diagnóstico.
Verificación realizada	Sobre la copia foodstore_tp2, dentro de BEGIN ... ROLLBACK primero (inspección) y luego con COMMIT definitivo: — R1: pedido para cliente inactivo → rechazado ("No se puede crear un pedido para el cliente 1"); cliente activo → aceptado. — R2: línea con producto inactivo → rechazada ("No se puede vender el producto 2"); activo → aceptada. — R3: cantidad 5000 > stock 20 → rechazada ("Stock insuficiente ... pide 5000 pero hay 20"); cantidad 3 ≤ stock 50 → aceptada. Se comprobó además que los 3 triggers quedaron registrados en pg_trigger.

Verificación humana

- Cada línea del diff de restricciones_integridad.sql fue leída y entendida antes de aplicarse (política: ningún script se ejecuta sin leerse).
- El DDL se aplicó primero en una transacción que se revirtió para inspeccionar, y recién después se confirmó con COMMIT sobre la copia.
- Las decisiones de diseño (usar triggers en vez de CHECK porque involucran dos tablas; FOR EACH ROW; BEFORE INSERT/UPDATE OF cantidad) fueron entendidas.

4. Parte 2 — Laboratorio de concurrencia con dos sesiones

Informe de Concurrencia — Parte 2

TP: 2 · Laboratorio de Concurrencia e IA (Unidad 1, Semana 2) Proyecto: Food Store · Motor: PostgreSQL 17.10 · Base de trabajo: `foodstore_tp2` (copia según protocolo) Sesiones: dos conexiones concurrentes de `psql` (Sesión A y Sesión B) sobre la misma base, según la consigna del TP.

Método (regla de la cátedra): cada escenario se reprodujo → se pidió una explicación a la IA → se verificó en el motor real repitiendo el experimento con el nivel de aislamiento propuesto → se registró si la IA acertó. La que vale es la que confirma el motor, no la que da el modelo.

Escenario 1 — Lectura no repetible

Cómo se reprodujo

Tabla objetivo: `producto` (fila `id = 2`, Coca, precio base 800.00).

- Sesión A (`concurrency/escenario1_no_repetible/sA_rc.sql`): inicia transacción con `SET TRANSACTION ISOLATION LEVEL READ COMMITTED`, lee `precio`, duerme 5 s, y vuelve a leer `precio` en la misma transacción.
- Sesión B (`sB1.sql`): tras 2 s, actualiza `precio = 850.00` y hace `COMMIT` mientras A duerme.

Qué se observó (salida real en el motor)

```
SESION A (READ COMMITTED)
>>> Lectura 1: precio = 800.00
>>> A espera 5 segundos mientras B actualiza y confirma...
>>> Lectura 2: precio = 850.00      <-- CAMBIO el valor en la misma transaccion
SESION B: UPDATE 1 / COMMIT / precio ahora es 850.00
```

Explicación de la IA

"Bajo `READ COMMITTED`, PostgreSQL toma un snapshot nuevo por cada sentencia. Los dos `SELECT` de la Sesión A son dos sentencias distintas y cada una ve una 'foto' distinta: la primera ve la versión con precio 800 y la segunda ve la versión posterior con precio 850 confirmada por B. Por eso el valor cambia dentro de la misma transacción: es una lectura no repetible. Con `REPEATABLE READ`, en cambio, la transacción toma un único snapshot al inicio y lo conserva hasta el `COMMIT`, por lo que ambas lecturas deberían ver el mismo 800 sin importar que B confirme el 850."

Verificación en el motor

Se repitió el experimento con `SESION A` en `SET TRANSACTION ISOLATION LEVEL REPEATABLE READ` (`sA_rr.sql`), con B haciendo la misma actualización a 850 + `COMMIT`:

```
SESION A (REPEATABLE READ)
>>> Lectura 1: precio = 800.00
>>> Lectura 2: precio = 800.00      <-- NO cambia (mismo snapshot)
SESION B: UPDATE 1 / COMMIT / precio ahora es 850.00
```

(Registrado en `salida_A_rr.txt`.)

Conclusión

La IA acertó. Bajo `READ COMMITTED` el valor cambió de 800 a 850 (lectura no repetible); bajo `REPEATABLE READ` se mantuvo en 800. El mecanismo es el MVCC y el nivel de aislamiento que resuelve el problema es `REPEATABLE READ` (snapshot único por transacción).

Escenario 2 — Lectura fantasma

Cómo se reprodujo

Tabla objetivo: producto filtrado por categoria_id = 1 (Pizzas, estado base con 1 producto).

- Sesión A (escenario2_fantasma/sA_rc.sql): BEGIN; SELECT count(*) FROM producto WHERE categoria_id = 1; → duerme 5 s → repite el mismo count(*).
- Sesión B (sB2.sql): tras 2 s, INSERT INTO producto (categoria_id=1, nombre='Fugazzeta', ...) y confirma.

Qué se observó (salida real del motor)

```
SESION A (READ COMMITTED)
>>> COUNT 1: 1
>>> A espera 5 s mientras B inserta una pizza nueva y confirma...
>>> COUNT 2: 2      <-- aparecio una fila NUEVA que cumple el WHERE
SESION B: INSERT 0 1 / SESION B confirmo insercion de Fugazzeta
```

Explicación de la IA

"Una lectura fantasma es una fila nueva que aparece en el resultado de una consulta repetida dentro de la misma transacción, porque otra sesión la insertó y confirmó en el medio. En READ COMMITTED, cada sentencia ve un snapshot distinto, así que el segundo count() incluye la 'Fugazzeta' y cambia de 1 a 2. El nivel REPEATABLE READ de PostgreSQL sí resuelve los fantasmas (su snapshot único cubre también los INSERT), a diferencia del estándar SQL."*

Verificación en el motor

Se repitió con SESION A en REPEATABLE READ (sA_rr.sql), B insertando la misma fila:

```
SESION A (REPEATABLE READ)
>>> COUNT 1: 1
>>> COUNT 2: 1      <-- NO ve la fila insertada por B (sin fantasma)
SESION B: INSERT 0 1 / confirmo
```

(Registrado en salida_A_rr.txt; la corrida READ COMMITTED quedó en salida_A_rc.txt, ambas en el directorio del escenario 2.)

Conclusión

La IA acertó. Bajo READ COMMITTED el count cambió de 1 a 2 (fantasma); bajo REPEATABLE READ se mantuvo en 1. En PostgreSQL, REPEATABLE READ elimina también los fantasmas; la anomalía aparece en READ COMMITTED.

Escenario 3 — Espera por bloqueo (FOR UPDATE)

Cómo se reprodujo

Tabla objetivo: producto (fila id = 1, Muzzarella).

- Sesión A (escenario3_bloqueo/sA.sql): BEGIN; SELECT ... FROM producto WHERE id = 1 FOR UPDATE; (toma bloqueo de fila), duerme 5 s, COMMIT.
- Sesión B (sB.sql): tras 2 s, SELECT ... FROM producto WHERE id = 1 FOR UPDATE; — debe quedar esperando hasta el COMMIT de A.

Qué se observó (salida real del motor, con timestamps)

```
SESION A: bloquea fila 1, duerme, COMMIT (libera el bloqueo)
SESION B:
  inicio_intento: 18:05:25
>>> B pide FOR UPDATE sobre fila 1 (debe esperar)
  antes   : 18:05:25
  (obtiene la fila DESPUES de que A hizo COMMIT)
  despues : 18:05:28      <-- espero 3 s hasta el COMMIT de A
```

```
>>> B obtuvo el bloqueo SOLO despues del COMMIT de A
```

Explicación de la IA

"Cuando una sesión ejecuta `SELECT ... FOR UPDATE`, obtiene un bloqueo de fila exclusivo y lo mantiene hasta el `COMMIT/ROLLBACK`. Si otra sesión pide `FOR UPDATE` sobre la misma fila, no recibe un error: queda esperando (en `wait_event = Lock`) hasta que la primera confirme y libere el bloqueo. Por defecto no hay timeout de espera de bloqueo de fila, así que la espera puede ser indefinida si la primera nunca confirma. Es el clásico bloqueo de fila de la Semana 2."

Verificación en el motor

La Sesión B pidió a las 18:05:25 y recién obtuvo el bloqueo a las 18:05:28, justo después del `COMMIT` de A. La espera de 3 s es evidencia directa del bloqueo de fila.

Conclusión

La IA acertó. La Sesión B quedó en espera por el bloqueo de fila `FOR UPDATE` de A y continuó solo después del `COMMIT`. No requiere cambiar nivel de aislamiento: es espera por bloqueo, y se resuelve cuando la sesión bloqueante confirma.

Escenario 4 — Interbloqueo real (opcional, nota adicional)

Cómo se reprodujo

Dos sesiones tomando filas en orden cruzado sobre `producto`, con sincronización para garantizar el ciclo:

- Sesión A (`escenario4_interbloqueo/sA.sql`): bloquea `producto 1`, espera la señal de B, e intenta bloquear `producto 2`.
- Sesión B (`sB.sql`): bloquea `producto 2`, avisa a A, e intenta bloquear `producto 1`.

Qué se observó (salida real del motor)

```
SESION A (victima):
psql:...sA.sql:21: ERROR:  se ha detectado un deadlock
DETALLE:  El proceso 18788 espera ShareLock en transaccion 807; bloqueado por proceso 9536.
El proceso 9536 espera ShareLock en transaccion 806; bloqueado por proceso 18788.
CONTEXTO:  mientras se bloqueaba la tupla (0,15) de la relacion "producto"
ROLLBACK (la transaccion de A se aborta con el error 40P01)
SESION B:
(completo su COMMIT sin error, su transaccion sobrevive al deadlock)
```

Explicación de la IA

"Un interbloqueo ocurre cuando dos transacciones se bloquean mutuamente: A tiene la fila 1 y pide la 2 mientras B tiene la 2 y pide la 1. Ninguna puede avanzar sola, así que se forma un ciclo en el grafo de espera. PostgreSQL detecta el ciclo (cada 1 s por defecto, `deadlock_timeout`) y aborta a una de las dos como víctima con el error `SQLSTATE 40P01`, liberando sus bloqueos para que la otra continúe. La prevención estándar es acceder siempre a los recursos en el mismo orden y mantener transacciones cortas."

Verificación en el motor

PostgreSQL abortó a la Sesión A con el error `40P01` ("se ha detectado un deadlock"), mostrando en el DETALLE el ciclo exacto (el proceso 18788 espera a 9536, y este a su vez espera a 18788). La Sesión B continuó y confirmó.

Conclusión

La IA acertó. El ciclo se formó con el orden cruzado de acceso y el motor lo resolvió abortando a una víctima (A). La solución de diseño es acceder a los recursos en un orden consistente (evitar el cruce) y mantener

transacciones cortas.

Resumen

Escenario	Anomalía / fenómeno	Nivel en el que aparece	Nivel/mecanismo que lo resuelve	¿La IA acertó?
1	Lectura no repetible	READ COMMITTED	REPEATABLE READ (snapshot único) / MVCC	Sí
2	Lectura fantasma	READ COMMITTED	REPEATABLE READ	Sí
3	Espera por bloqueo (FOR UPDATE)	cualquier nivel (bloqueo de fila)	COMMIT de la sesión bloqueante	Sí
4	Interbloqueo (40P01)	orden cruzado de bloqueos	orden consistente de acceso + transacciones cortas	Sí

Observación: en los cuatro escenarios la explicación de la IA se confirmó exactamente en el motor real. No hubo discrepancias que documentar, pero el método aplicado fue el de la cátedra: reproducir, pedir explicación, verificar en el motor y registrar el resultado.

4.1. Evidencia real del motor (salidas de las sesiones)

Escenario 1 — Lectura no repetible: Sesión A en READ COMMITTED (salida_A_rc.txt)

```
BEGIN
SET
      quien          |          momento
-----+-----
SESSION A (READ COMMITTED) | 2026-09-03 18:03:57.74181-03
(1 fila)

>>> Lectura 1 (inicio de transaccion):
  id | nombre  | precio
-----+-----
   2 | Coca 1.5L | 800.00
(1 fila)

>>> A espera 5 segundos mientras B actualiza y confirma...
pg_sleep
-----

(1 fila)

>>> Lectura 2 (misma transaccion, despues del COMMIT de B):
  id | nombre  | precio
-----+-----
   2 | Coca 1.5L | 850.00
(1 fila)

COMMIT
>>> FIN SESION A (RR: verificar resultado en reporte);
```

Escenario 1 — Sesión A en REPEATABLE READ: no cambia (salida_A_rr.txt)

```
BEGIN
SET
      quien          |          momento
```

```

-----+-----
SESION A (REPEATABLE READ) | 2026-09-03 18:04:16.073316-03
(1 fila)

>>> Lectura 1 (inicio de transaccion):
  id | nombre  | precio
-----+-----
   2 | Coca 1.5L | 800.00
(1 fila)

>>> A espera 5 segundos mientras B actualiza y confirma...
pg_sleep
-----

(1 fila)

>>> Lectura 2 (misma transaccion, despues del COMMIT de B):
  id | nombre  | precio
-----+-----
   2 | Coca 1.5L | 800.00
(1 fila)

COMMIT
>>> FIN SESION A (RR);

```

Escenario 1 — Sesión B: UPDATE y COMMIT (salida_B.txt)

```

>>> SESION B inicia con 1.5s de retardo
pg_sleep
-----

(1 fila)

BEGIN
UPDATE 1
COMMIT
>>> SESION B confirmo: precio del producto 2 ahora es 850.00;
  id | nombre  | precio
-----+-----
   2 | Coca 1.5L | 850.00
(1 fila)

```

Escenario 2 — Lectura fantasma: COUNT 1 → 2 en READ COMMITTED (salida_A_rc.txt)

```

BEGIN
SET
>>> COUNT 1 (inicio):
  pizzas
-----
   1
(1 fila)

>>> A espera 5s mientras B inserta una pizza nueva y confirma...
pg_sleep
-----

(1 fila)

>>> COUNT 2 (misma transaccion, despues de COMMIT de B):
  pizzas
-----
   2
(1 fila)

COMMIT

```

Escenario 2 — COUNT 1 → 1 en REPEATABLE READ, sin fantasma (salida_A_rr.txt)

```
BEGIN
SET
>>> COUNT 1 (inicio):
  pizzas
-----
      1
(1 fila)

>>> A espera 5s mientras B inserta una pizza nueva y confirma...
  pg_sleep
-----

(1 fila)

>>> COUNT 2 (misma transaccion, despues de COMMIT de B):
  pizzas
-----
      1
(1 fila)

COMMIT
```

Escenario 3 — Espera por bloqueo: Sesión A bloquea y hace COMMIT (salida_A.txt)

```
BEGIN
>>> A bloquea fila producto 1 con FOR UPDATE y se queda 5s:
  id |  nombre  | stock
-----+-----+-----
   1 | Muzzarella |    20
(1 fila)

  pg_sleep
-----

(1 fila)

COMMIT
>>> A hizo COMMIT: libero el bloqueo de la fila 1;
```

Escenario 3 — Sesión B: espera 18:05:25 → obtiene a 18:05:28 (salida_B.txt)

```
  inicio_intento
-----
18:05:25
(1 fila)

  inicio
-----
18:05:25
(1 fila)

>>> B pide FOR UPDATE sobre la fila 1 (debe esperar a que A haga COMMIT):
  antes
-----
18:05:25
(1 fila)

  id |  nombre  | stock
-----+-----+-----
   1 | Muzzarella |    20
(1 fila)

  despues
```



```

-----
18:05:28
(1 fila)

>>> B obtuvo el bloqueo SOLO despues del COMMIT de A (espera comprobada);

```

Escenario 4 — Interbloqueo: Sesión A víctima, error 40P01 (salida_A.txt)

```

BEGIN
>>> A bloquea producto 1:
  id
----
  1
(1 fila)

>>> A espera (hasta 20s) que B bloquee producto 2:
DO
>>> A (ya con fila 1) intenta fila 2 que B tiene -> forma ciclo:
ROLLBACK
psql:C:/UTN/3er semestre/Bases de Datos II/UNIDAD 1/concurrencia/escenario4_interbloqueo/sA.sql:21: ERROR:  se ha detectado un ciclo de esperas
DETALLE:  El proceso 18788 espera ShareLock en transacción 807; bloqueado por proceso 9536.
El proceso 9536 espera ShareLock en transacción 806; bloqueado por proceso 18788.
SUGERENCIA:  Vea el registro del servidor para obtener detalles de las consultas.
CONTEXTO:  mientras se bloqueaba la tupla (0,15) de la relación «producto»

```

Escenario 4 — Sesión B sobrevive y hace COMMIT (salida_B.txt)

```

>>> SESION B espera 1s y arranca:
  pg_sleep
-----

(1 fila)

BEGIN
>>> B bloquea producto 2:
  id
----
  2
(1 fila)

UPDATE 1
>>> B aviso a A; ahora intenta bloquear producto 1 (A lo tiene):
  id
----
  1
(1 fila)

COMMIT

```

4.2. Declaración de Uso de IA — Parte 2

Declaración de Uso de IA (DUIA) — Parte 2 · Concurrencia con dos sesiones

TP: 2 · Concurrencia e IA (Parte 2) Motor: PostgreSQL 17.10 · Base de trabajo: foodstore_tp2 (copia)

Campo	Completar
Herramienta	OpenCode (modelo de línea big-pickle)

Spec o prompt utilizado	"Generá y ejecutá, sobre el esquema FoodStore con dos sesiones concurrentes de psql, la reproducción de: (1) lectura no repetible en READ COMMITTED y su eliminación en REPEATABLE READ; (2) lectura fantasma en READ COMMITTED y su eliminación en REPEATABLE READ; (3) espera por bloqueo con SELECT FOR UPDATE; y (4, opcional) interbloqueo real con orden cruzado (error 40P01). Para cada escenario, guardá la explicación que darías y verificala en el motor."
Qué generó	Los scripts por sesión en <code>concurrency/escenarioN/</code> (<code>sA_rc.sql</code> , <code>sA_rr.sql</code> , <code>sB*.sql</code> , <code>sA.sql</code> , <code>sB.sql</code>), los orquestadores <code>runN.ps1</code> que lanzan las dos sesiones en paralelo, y el archivo <code>informe_concurrency.md</code> .
Qué se aceptó	El enfoque de simular dos sesiones con dos procesos <code>psql</code> lanzados en paralelo vía <code>Start-Process</code> , con <code>pg_sleep()</code> y una señal (<code>sync_deadlock</code>) para coordinar el interbloqueo. Los 4 scripts de sesión y el informe tal como se generaron.
Qué se modificó o descartado, y por qué	1) El interbloqueo se intentó primero sin sincronización y no se formaba el ciclo por timing; se agregó una tabla <code>sync_deadlock</code> y un bucle de espera en la Sesión A para garantizar que B ya tuviera la fila 2 antes del cruce. 2) En el escenario 2, las corridas coincidentes se aislaron borrando la fila 'Fugazzeta' entre repeticiones para no contaminar el <code>count</code> . 3) En el escenario 1/2 se restauró <code>precio = 800</code> entre corridas para mantener el estado base reproducible.
Verificación realizada	Cada escenario se ejecutó realmente y se capturó la salida del motor en <code>salida_*.txt</code> : RC mostró 800→850 y count 1→2; RR mantuvo 800 y 1; el bloqueo B esperó de 18:05:25 a 18:05:28; el interbloqueo abortó a la sesión A con error 40P01. Todas las explicaciones de la IA se confirmaron en el motor (ver informe).

Verificación humana

- Los scripts se leyeron antes de ejecutarse (política de la cátedra).
- Las salidas reales del motor (`salida_*.txt`) respaldan cada afirmación del `informe_concurrency.md`.
- Se aplicó la regla: la explicación que vale es la que confirma el motor, no la que da el modelo.

5. Parte 3 — Lectura crítica de scripts de IA

5.1. Ejercicio de lectura crítica (análisis y corrección)

Ejercicio de Lectura Crítica — Parte 3

TP: 2 · Concurrency e IA (Parte 3) Regla de fondo: ningún script generado por IA se ejecuta sin leerse antes.

El objetivo de esta parte es reconocer, sobre casos reales, por qué ningún script se corre sin leerse: la sintaxis puede ser correcta y la intención razonable, pero un `WHERE` que falta o una subconsulta con `NULL` puede tocar filas que no eran las que se quería. Estos son los dos scripts "generados para dar de baja registros vencidos" de la consigna, sobre el esquema genérico de la cátedra.

Script 1

```
-- Generado para: dar de baja las funciones de películas retiradas de cartel
UPDATE funcion
SET activa = FALSE;
```

Qué filas afectaría realmente

Tal como está escrito, el `UPDATE` sin cláusula `WHERE` modifica todas las filas de la tabla `funcion`: tanto las de películas retiradas de cartel como las películas en cartel y las funciones ya pasadas. Es decir, afecta el 100 % de la tabla (todas las funciones existentes pasan a `activa = FALSE`).

Por qué eso no coincide con la consigna

La consigna dice: dar de baja las funciones de películas retiradas de cartel (un subconjunto). El script sin `WHERE` da de baja todo, no el subconjunto pedido. El efecto es idéntico al caso Replit de la teoría: un agente con el código "correcto" pero la condición ausente borró/afectó mucho más de lo solicitado.

En el esquema del proyecto (FoodStore), este mismo error aparecería por ejemplo como

```
UPDATE producto SET activo = FALSE; -- daría de baja TODOS los productos, no solo los retirados
```

Donde a simple vista parece trivial pero afecta toda la tabla.

Versión corregida

Hay que limitar el `UPDATE` con un `WHERE` que identifique exactamente el subconjunto "funciones de películas retiradas de cartel". En el esquema genérico de cátedra, la película vive en `funcion.pelicula_id` con el estado en otra tabla (o en la misma con un atributo de película); asumimos que `funcion` referencia la película por `pelicula_id` y que la película tiene `en_cartel`:

```
UPDATE funcion
SET activa = FALSE
WHERE pelicula_id IN (
    SELECT id FROM pelicula WHERE en_cartel = FALSE
);
```

En el proyecto FoodStore, el equivalente corregido para "dar de baja solo los productos de una categoría retirada" sería: `sql UPDATE producto SET activo = FALSE WHERE categoria_id IN (SELECT id FROM categoria WHERE activo = FALSE); ```

Regla práctica: un `UPDATE` (o `DELETE`) de producción sin `WHERE` es una señal de alarma. Si la consigna menciona un subconjunto, debe haber `WHERE`.

Script 2

```
-- Generado para: limpiar las categorías sin productos asociados
DELETE FROM categoria
WHERE id NOT IN (SELECT categoria_id FROM producto);
```

Qué filas afectaría realmente

La intención es borrar las categorías que no tienen ningún producto. Pero la subconsulta `SELECT categoria_id FROM producto` puede devolver `NULL` — por ejemplo, si alguna fila de `producto` tuviera `categoria_id` nulo (en este proyecto la FK lo prohíbe, pero en el esquema genérico la columna puede ser nullable). El operador `NOT IN` es delicado con `NULL`: si el conjunto de la subconsulta contiene aunque sea un `NULL`, la condición `id NOT IN (...)` nunca es cierta (siempre es `NULL`), y el `DELETE` no borra ninguna fila.

Efecto real del script:

- Si `producto.categoria_id` es nullable y existe al menos un `NULL` → el `DELETE` afecta 0 filas (no limpia nada).
- Si ninguna fila tiene `NULL` → sí borra las categorías sin productos (lo deseado).

Por qué eso no coincide con la consigna

Depende del estado de los datos:

- Con un `NULL` en `producto.categoria_id`: la consigna pide "limpiar las categorías sin productos", pero el script no borra nada. Además, de forma insidiosa, la categoría sin productos queda intacta (no cumple la consigna) en silencio, sin error.
- Sin `NULL`: funciona, pero el comportamiento correcto depende de un detalle de datos que el `NOT IN` oculta.

La regla de oro es: `NOT IN` es peligroso con `NULL`; la versión segura es usar `NOT EXISTS`, que maneja bien los `NULL`.

Versión corregida (manejo de NULL)

Se usa `NOT EXISTS` para que la lógica sea correcta aunque haya `NULL` en `producto.categoria_id`:

```
DELETE FROM categoria c
WHERE NOT EXISTS (
    SELECT 1 FROM producto p WHERE p.categoria_id = c.id
);
```

Con `NOT EXISTS`, los `NULL` de `categoria_id` simplemente no matchean ningún `c.id`, así que se borran exactamente las categorías sin productos, sin importar si hay nulos. Es la recomendación estándar frente a `NOT IN`.

Nota sobre el proyecto FoodStore: en `schema.sql` la FK `producto.categoria_id` es `NOT NULL`, así que en rigor no podría haber `NULL` y el `NOT IN` funcionaría. Pero en un esquema genérico —y por defensa del código— conviene igual usar `NOT EXISTS`, porque la restricción puede no existir en todas las tablas y elimina el riesgo de raíz. Además, en FoodStore las categorías usan baja lógica (activo), no borrado físico, así que la acción correcta del dominio sería `UPDATE categoria SET activo = FALSE WHERE NOT EXISTS (...)`.

Patrón común de los casos reales (teoría)

En los incidentes documentados (Replit, Gemini CLI, PocketOS, confusión de entorno) el código era sintácticamente válido y la intención razonable. Lo que falló fue el paso del humano: confirmar contra qué base se corría, leer el efecto real del comando, y no confiar en el reporte del agente. Es exactamente lo que el protocolo de la Parte 0 obliga a interponer:

1. Copia — el error no toca nada real.
2. Transacción (`BEGIN ... ROLLBACK`) — permite inspeccionar las filas afectadas antes de confirmar.
3. Respaldo — para cuando ni siquiera la transacción alcanza.

Y, sobre los scripts, este ejercicio demuestra el caso concreto: un `UPDATE` sin `WHERE` (Script 1) y un `NOT IN` con `NULL` (Script 2) son dos formas en las que la sintaxis correcta produce un efecto distinto del consignado.

5.2. Declaración de Uso de IA — Parte 3

Declaración de Uso de IA (DUIA) — Parte 3 · Lectura crítica

TP: 2 · Concurrencia e IA (Parte 3) Regla de fondo: ningún script generado por IA se ejecuta sin leerse antes.

Campo	Completar
Herramienta	OpenCode (modelo de línea big-pickle)
Spec o prompt utilizado	"Enunciado Parte 3 de la consigna: para cada uno de estos dos scripts 'generados para dar de baja registros vencidos', identifiqué qué filas afectaría realmente tal como está escrito, por qué eso no coincide con la consigna que dice cumplir, y reescribílo corregido (con su <code>WHERE</code> , o con el manejo correcto de <code>NULL</code> en la subconsulta, según corresponda)."
Qué generó	El análisis de los dos scripts (<code>UPDATE funcion SET activa = FALSE; sin WHERE y DELETE FROM categoria WHERE id NOT IN (...)</code>), la explicación del efecto real de cada uno y sus versiones corregidas, incluidas en <code>ejercicio_lectura_critica.md</code> .
Qué se aceptó	El diagnóstico en ambos casos: (1) el <code>UPDATE</code> sin <code>WHERE</code> afecta el 100 % de las filas y se corrige con un <code>WHERE</code> que identifique el subconjunto pedido; (2) el <code>NOT IN</code> es peligroso con <code>NULL</code> (si la subconsulta devuelve un <code>NULL</code> , la condición nunca es verdadera y el <code>DELETE</code> no borra nada), y se corrige con <code>NOT EXISTS</code> .
Qué se modificó o descartado, y por qué	La corrección se adaptó al dominio del proyecto FoodStore: como FoodStore usa baja lógica (<code>activo</code>), el equivalente real de "dar de baja" no es <code>DELETE</code> sino <code>UPDATE ... SET activo = FALSE</code> . Además se dejó el análisis sobre el esquema genérico de cátedra (donde <code>producto.categoria_id</code> puede ser nullable) y se aclaró que en <code>schema.sql</code> la FK es <code>NOT NULL</code> , pero que por defensa del código conviene <code>NOT EXISTS</code> igual.
Verificación realizada	Se trazó el efecto de cada script contra el esquema (<code>schema.sql</code>): el <code>UPDATE</code> sin <code>WHERE</code> recorre toda la tabla; el <code>NOT IN</code> depende de la presencia de <code>NULL</code> en <code>producto.categoria_id</code> , que la FK <code>NOT NULL</code> de FoodStore excluye pero el esquema genérico no. Con <code>NOT EXISTS</code> la semántica es correcta en ambos escenarios. La conclusión coincide con el patrón común de los incidentes documentados (Replit, Gemini CLI, PocketOS, confusión de entorno).

Verificación humana

- El análisis fue revisado y entendido línea por línea ANTES de considerarlo correcto (política de la cátedra).
- La guía de la consigna pide entregar "qué filas afectaría realmente cada uno, por qué no coincide y la versión corregida"; el archivo `ejercicio_lectura_critica.md` cubre los tres campos para ambos scripts.

- Ninguno de estos scripts se ejecutó jamás contra la base: se analizaron como ejercicio de lectura crítica, sin tocar `foodstore` ni `foodstore_tp2`.

6. Historial de commits (git log)

```
aba8076 Fixes concurrencia: evidencia separada RC/RR del escenario 2 (fantasma 1->2 RC, 1->1 RR), correccion coherencia
b814865 Fixes: evidencia del deadlock 40P01 anexada a salida_A.txt y alineada con el informe; orquestador combina stderr
e9e2690 Concurrencia: reprod y verificacion en motor de lectura no repetible, lectura fantasma, espera por bloqueo e int
991dcf0 Integridad: triggers que impiden pedidos a clientes inactivos, venta de productos inactivos y stock sobrevendido
8579a26 Semana 1 y Parte 0 TP2: esquema FoodStore + protocolo de seguridad
```

Cada commit corresponde a un entregable del TP: la Parte 0 y el esquema (8579a26), las restricciones de la Parte 1 (991dcf0), la concurrencia con su informe y DUJA de la Parte 2 (e3c697b) y los fixes de evidencia de la Parte 2 (b9d1aa3 y abf07bf). Ningún commit es un volcado genérico: cada mensaje describe la regla de negocio garantizada o la anomalía verificada.

7. Checklist final antes de entregar

- ☐ protocolo_seguridad.md commiteado y aplicado en cada paso posterior.
- ☐ Cada script generado por IA fue leído línea por línea antes de aplicarse.
- ☐ Cada aplicación sobre la copia de trabajo pasó primero por BEGIN ... ROLLBACK.
- ☐ Las tres DUIA (Parte 1, Parte 2, Parte 3) están completas.
- ☐ Podés explicar, sin mirar el archivo, qué hace cada línea que commiteaste.

Estado: todas las casillas se cumplen en esta entrega. La DUIA de la Parte 3 se agregó al compilar este PDF; las DUIA de las Partes 1 y 2 estaban versionadas junto con sus scripts.